

```

# dataset ('balraj98/horse2zebra-dataset')
# models ('balraj98/cyclegan-translating-horses-zebras-pytorch')

from zipfile import ZipFile

# Specify the path to your zip file
zip_path = '/content/cyclegan-translating-horses-zebras-pytorch.zip'

# Specify the directory where you want to extract the files
extract_path = '/content'

# Open the zip file and extract its contents
with ZipFile(zip_path, 'r') as zip_ref:
    zip_ref.extractall(extract_path)

print("Files extracted successfully!")
Files extracted successfully!

import shutil

# Specify the path to the folder you want to delete
folder_path = '/content/horse2zebra-dataset'

# Use shutil.rmtree() to delete the folder and its contents
shutil.rmtree(folder_path)

print(f"Folder '{folder_path}' deleted successfully!")
Folder '/content/horse2zebra-dataset' deleted successfully!

```

Introduction

In this notebook we use [CycleGAN](#) to convert Horse Images to Zebra ones using [Horse2zebra Dataset](#).

Image Source: Unpaired Image-to-Image Translation using Cycle-Consistent Adversarial Networks [C. Jun-Yan Zhu et al.]

Libraries ↓

```

import numpy as np
import pandas as pd
import os, math, sys
import time, datetime
import glob, itertools
import argparse, random

import torch
import torch.nn as nn

```

```

import torch.nn.functional as F
from torchvision import datasets
from torch.autograd import Variable
from torchvision.models import vgg19
import torchvision.transforms as transforms
from torch.utils.data import DataLoader, Dataset
from torchvision.utils import save_image, make_grid

import plotly
from scipy import signal
import plotly.express as px
import plotly.graph_objects as go
import matplotlib.pyplot as plt

from PIL import Image
from tqdm import tqdm_notebook as tqdm
from sklearn.model_selection import train_test_split

random.seed(42)
import warnings
warnings.filterwarnings("ignore")

```

Settings

```

# path to pre-trained models
pretrained_model_path = "/content/cyclegan-translating-horses-zebras-
pytorch/saved_models"
# epoch to start training from
epoch_start = 25
# number of epochs of training
n_epochs = 60
# name of the dataset
dataset_path = "/content/horse2zebra-dataset"
# size of the batches"
batch_size = 4
# adam: learning rate
lr = 0.0001
# adam: decay of first order momentum of gradient
b1 = 0.5
# adam: decay of first order momentum of gradient
b2 = 0.999
# epoch from which to start lr decay
decay_epoch = 1
# number of cpu threads to use during batch generation
n_workers = 8
# size of image height
img_height = 256
# size of image width
img_width = 256
# number of image channels

```

```

channels = 3
# interval between saving generator outputs
sample_interval = 100
# interval between saving model checkpoints
checkpoint_interval = -1
# number of residual blocks in generator
n_residual_blocks = 9
# cycle loss weight
lambda_cyc = 10.0
# identity loss weight
lambda_id = 5.0
# Development / Debug Mode
debug_mode = False

# Create images and checkpoint directories
os.makedirs("images", exist_ok=True)
os.makedirs("saved_models", exist_ok=True)

```

Define Utilities

```

def to_rgb(image):
    rgb_image = Image.new("RGB", image.size)
    rgb_image.paste(image)
    return rgb_image

class ReplayBuffer:
    def __init__(self, max_size=50):
        assert max_size > 0
        self.max_size = max_size
        self.data = []

    def push_and_pop(self, data):
        to_return = []
        for element in data.data:
            element = torch.unsqueeze(element, 0)
            if len(self.data) < self.max_size:
                self.data.append(element)
                to_return.append(element)
            else:
                if random.uniform(0, 1) > 0.5:
                    i = random.randint(0, self.max_size - 1)
                    to_return.append(self.data[i].clone())
                    self.data[i] = element
                else:
                    to_return.append(element)
        return Variable(torch.cat(to_return))

```

Define Dataset Class

```
class ImageDataset(Dataset):
    def __init__(self, root, transforms_=None, unaligned=False,
mode="train"):
        self.transform = transforms.Compose(transforms_)
        self.unaligned = unaligned

        self.files_A = sorted(glob.glob(os.path.join(root, f"{mode}A")
+ "/*.*.png"))
        self.files_B = sorted(glob.glob(os.path.join(root, f"{mode}B")
+ "/*.*.png"))
        if debug_mode:
            self.files_A = self.files_A[:100]
            self.files_B = self.files_B[:100]

    def __getitem__(self, index):
        image_A = Image.open(self.files_A[index % len(self.files_A)])

        if self.unaligned:
            image_B = Image.open(self.files_B[random.randint(0,
len(self.files_B) - 1)])
        else:
            image_B = Image.open(self.files_B[index %
len(self.files_B)])

        # Convert grayscale images to rgb
        if image_A.mode != "RGB":
            image_A = to_rgb(image_A)
        if image_B.mode != "RGB":
            image_B = to_rgb(image_B)

        item_A = self.transform(image_A)
        item_B = self.transform(image_B)
        return {"A": item_A, "B": item_B}

    def __len__(self):
        return max(len(self.files_A), len(self.files_B))
```

Get Train/Test Dataloaders

```
# Image transformations
transforms_ = [
    transforms.Resize(int(img_height * 1.12), Image.BICUBIC),
    transforms.RandomCrop((img_height, img_width)),
    transforms.RandomHorizontalFlip(),
    transforms.ToTensor(),
    transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5)),
]

# Training data loader
```

```

train_dataloader = DataLoader(
    ImageDataset(f"{dataset_path}", transforms_=transforms_,
unaligned=True),
    batch_size=batch_size,
    shuffle=True,
    num_workers=n_workers,
)
# Test data loader
test_dataloader = DataLoader(
    ImageDataset(f"{dataset_path}", transforms_=transforms_,
unaligned=True, mode="test"),
    batch_size=1,
    shuffle=True,
    num_workers=1,
)

```

Image Source: Unpaired Image-to-Image Translation using Cycle-Consistent Adversarial Networks [C. Jun-Yan Zhu et al.]

Define Model Classes

```

def weights_init_normal(m):
    classname = m.__class__.__name__
    if classname.find("Conv") != -1:
        torch.nn.init.normal_(m.weight.data, 0.0, 0.02)
        if hasattr(m, "bias") and m.bias is not None:
            torch.nn.init.constant_(m.bias.data, 0.0)
    elif classname.find("BatchNorm2d") != -1:
        torch.nn.init.normal_(m.weight.data, 1.0, 0.02)
        torch.nn.init.constant_(m.bias.data, 0.0)

class ResidualBlock(nn.Module):
    def __init__(self, in_features):
        super(ResidualBlock, self).__init__()

        self.block = nn.Sequential(
            nn.ReflectionPad2d(1),
            nn.Conv2d(in_features, in_features, 3),
            nn.InstanceNorm2d(in_features),
            nn.ReLU(inplace=True),
            nn.ReflectionPad2d(1),
            nn.Conv2d(in_features, in_features, 3),
            nn.InstanceNorm2d(in_features),
        )

    def forward(self, x):
        return x + self.block(x)

```

```

class GeneratorResNet(nn.Module):
    def __init__(self, input_shape, num_residual_blocks):
        super(GeneratorResNet, self).__init__()

        channels = input_shape[0]

        # Initial convolution block
        out_features = 64
        model = [
            nn.ReflectionPad2d(channels),
            nn.Conv2d(channels, out_features, 7),
            nn.InstanceNorm2d(out_features),
            nn.ReLU(inplace=True),
        ]
        in_features = out_features

        # Downsampling
        for _ in range(2):
            out_features *= 2
            model += [
                nn.Conv2d(in_features, out_features, 3, stride=2,
padding=1),
                nn.InstanceNorm2d(out_features),
                nn.ReLU(inplace=True),
            ]
            in_features = out_features

        # Residual blocks
        for _ in range(num_residual_blocks):
            model += [ResidualBlock(out_features)]

        # Upsampling
        for _ in range(2):
            out_features //= 2
            model += [
                nn.Upsample(scale_factor=2),
                nn.Conv2d(in_features, out_features, 3, stride=1,
padding=1),
                nn.InstanceNorm2d(out_features),
                nn.ReLU(inplace=True),
            ]
            in_features = out_features

        # Output layer
        model += [nn.ReflectionPad2d(channels),
nn.Conv2d(out_features, channels, 7), nn.Tanh()]

        self.model = nn.Sequential(*model)

    def forward(self, x):

```

```

        return self.model(x)

class Discriminator(nn.Module):
    def __init__(self, input_shape):
        super(Discriminator, self).__init__()

        channels, height, width = input_shape

        # Calculate output shape of image discriminator (PatchGAN)
        self.output_shape = (1, height // 2 ** 4, width // 2 ** 4)

        def discriminator_block(in_filters, out_filters,
                                normalize=True):
            """Returns downsampling layers of each discriminator
            block"""
            layers = [nn.Conv2d(in_filters, out_filters, 4, stride=2,
                                padding=1)]
            if normalize:
                layers.append(nn.InstanceNorm2d(out_filters))
            layers.append(nn.LeakyReLU(0.2, inplace=True))
            return layers

        self.model = nn.Sequential(
            *discriminator_block(channels, 64, normalize=False),
            *discriminator_block(64, 128),
            *discriminator_block(128, 256),
            *discriminator_block(256, 512),
            nn.ZeroPad2d((1, 0, 1, 0)),
            nn.Conv2d(512, 1, 4, padding=1)
        )

    def forward(self, img):
        return self.model(img)

```

Train CycleGAN

```

# Losses
criterion_GAN = torch.nn.MSELoss()
criterion_cycle = torch.nn.L1Loss()
criterion_identity = torch.nn.L1Loss()

cuda = torch.cuda.is_available()

input_shape = (channels, img_height, img_width)

# Initialize generator and discriminator
G_AB = GeneratorResNet(input_shape, n_residual_blocks)
G_BA = GeneratorResNet(input_shape, n_residual_blocks)
D_A = Discriminator(input_shape)

```

```

D_B = Discriminator(input_shape)

if cuda:
    G_AB = G_AB.cuda()
    G_BA = G_BA.cuda()
    D_A = D_A.cuda()
    D_B = D_B.cuda()
    criterion_GAN.cuda()
    criterion_cycle.cuda()
    criterion_identity.cuda()

if epoch_start != 0:
    # Load pretrained models

G_AB.load_state_dict(torch.load(f"{pretrained_model_path}/G_AB.pth"))
G_BA.load_state_dict(torch.load(f"{pretrained_model_path}/G_BA.pth"))
D_A.load_state_dict(torch.load(f"{pretrained_model_path}/D_A.pth"))
D_B.load_state_dict(torch.load(f"{pretrained_model_path}/D_B.pth"))
else:
    # Initialize weights
    G_AB.apply(weights_init_normal)
    G_BA.apply(weights_init_normal)
    D_A.apply(weights_init_normal)
    D_B.apply(weights_init_normal)

# Optimizers
optimizer_G = torch.optim.Adam(
    itertools.chain(G_AB.parameters(), G_BA.parameters()), lr=lr,
    betas=(b1, b2)
)
optimizer_D_A = torch.optim.Adam(D_A.parameters(), lr=lr, betas=(b1,
b2))
optimizer_D_B = torch.optim.Adam(D_B.parameters(), lr=lr, betas=(b1,
b2))

Tensor = torch.cuda.FloatTensor if cuda else torch.Tensor

# Buffers of previously generated samples
fake_A_buffer = ReplayBuffer()
fake_B_buffer = ReplayBuffer()

train_counter = []
train_losses_gen, train_losses_id, train_losses_gan, train_losses_cyc
= [], [], [], []
train_losses_disc, train_losses_disc_a, train_losses_disc_b = [], [],
[]

```



```

test_counter = [2*idx*len(train_dataloader.dataset) for idx in
range(epoch_start+1, n_epochs+1)]
test_losses_gen, test_losses_disc = [], []

for epoch in range(epoch_start, n_epochs):

    ##### Training
    loss_gen = loss_id = loss_gan = loss_cyc = 0.0
    loss_disc = loss_disc_a = loss_disc_b = 0.0
    tqdm_bar = tqdm(train_dataloader, desc=f'Training Epoch {epoch} ',
total=int(len(train_dataloader)))
    for batch_idx, batch in enumerate(tqdm_bar):

        # Set model input
        real_A = Variable(batch["A"].type(Tensor))
        real_B = Variable(batch["B"].type(Tensor))
        # Adversarial ground truths
        valid = Variable(Tensor(np.ones((real_A.size(0),
*D_A.output_shape))), requires_grad=False)
        fake = Variable(Tensor(np.zeros((real_A.size(0),
*D_A.output_shape))), requires_grad=False)

        ### Train Generators
        G_AB.train()
        G_BA.train()
        optimizer_G.zero_grad()
        # Identity loss
        loss_id_A = criterion_identity(G_BA(real_A), real_A)
        loss_id_B = criterion_identity(G_AB(real_B), real_B)
        loss_identity = (loss_id_A + loss_id_B) / 2
        # GAN loss
        fake_B = G_AB(real_A)
        loss_GAN_AB = criterion_GAN(D_B(fake_B), valid)
        fake_A = G_BA(real_B)
        loss_GAN_BA = criterion_GAN(D_A(fake_A), valid)
        loss_GAN = (loss_GAN_AB + loss_GAN_BA) / 2
        # Cycle loss
        recov_A = G_BA(fake_B)
        loss_cycle_A = criterion_cycle(recov_A, real_A)
        recov_B = G_AB(fake_A)
        loss_cycle_B = criterion_cycle(recov_B, real_B)
        loss_cycle = (loss_cycle_A + loss_cycle_B) / 2
        # Total loss
        loss_G = lambda_id * loss_identity + loss_GAN + lambda_cyc *
loss_cycle
        loss_G.backward()
        optimizer_G.step()

        ### Train Discriminator-A
        D_A.train()

```

```

optimizer_D_A.zero_grad()
# Real loss
loss_real = criterion_GAN(D_A(real_A), valid)
# Fake loss (on batch of previously generated samples)
fake_A_ = fake_A_buffer.push_and_pop(fake_A)
loss_fake = criterion_GAN(D_A(fake_A_.detach()), fake)
# Total loss
loss_D_A = (loss_real + loss_fake) / 2
loss_D_A.backward()
optimizer_D_A.step()

### Train Discriminator-B
D_B.train()
optimizer_D_B.zero_grad()
# Real loss
loss_real = criterion_GAN(D_B(real_B), valid)
# Fake loss (on batch of previously generated samples)
fake_B_ = fake_B_buffer.push_and_pop(fake_B)
loss_fake = criterion_GAN(D_B(fake_B_.detach()), fake)
# Total loss
loss_D_B = (loss_real + loss_fake) / 2
loss_D_B.backward()
optimizer_D_B.step()
loss_D = (loss_D_A + loss_D_B) / 2

### Log Progress
loss_gen += loss_G.item(); loss_id += loss_identity.item();
loss_gan += loss_GAN.item(); loss_cyc += loss_cycle.item()
loss_disc += loss_D.item(); loss_disc_a += loss_D_A.item();
loss_disc_b += loss_D_B.item()
train_counter.append(2*(batch_idx*batch_size + real_A.size(0)
+ epoch*len(train_dataloader.dataset)))
train_losses_gen.append(loss_G.item());
train_losses_id.append(loss_identity.item());
train_losses_gan.append(loss_GAN.item());
train_losses_cyc.append(loss_cycle.item())
train_losses_disc.append(loss_D.item());
train_losses_disc_a.append(loss_D_A.item());
train_losses_disc_b.append(loss_D_B.item())
tqdm_bar.set_postfix(Gen_loss=loss_gen/(batch_idx+1),
identity=loss_id/(batch_idx+1), adv=loss_gan/(batch_idx+1),
cycle=loss_cyc/(batch_idx+1),
Disc_loss=loss_disc/(batch_idx+1),
disc_a=loss_disc_a/(batch_idx+1), disc_b=loss_disc_b/(batch_idx+1))

#### Testing
loss_gen = loss_id = loss_gan = loss_cyc = 0.0
loss_disc = loss_disc_a = loss_disc_b = 0.0
tqdm_bar = tqdm(test_dataloader, desc=f'Testing Epoch {epoch} ',
total=int(len(test_dataloader)))

```

```

for batch_idx, batch in enumerate(tqdm_bar):

    # Set model input
    real_A = Variable(batch["A"].type(Tensor))
    real_B = Variable(batch["B"].type(Tensor))
    # Adversarial ground truths
    valid = Variable(Tensor(np.ones((real_A.size(0),
*D_A.output_shape))), requires_grad=False)
    fake = Variable(Tensor(np.zeros((real_A.size(0),
*D_A.output_shape))), requires_grad=False)

    ### Test Generators
    G_AB.eval()
    G_BA.eval()
    # Identity loss
    loss_id_A = criterion_identity(G_BA(real_A), real_A)
    loss_id_B = criterion_identity(G_AB(real_B), real_B)
    loss_identity = (loss_id_A + loss_id_B) / 2
    # GAN loss
    fake_B = G_AB(real_A)
    loss_GAN_AB = criterion_GAN(D_B(fake_B), valid)
    fake_A = G_BA(real_B)
    loss_GAN_BA = criterion_GAN(D_A(fake_A), valid)
    loss_GAN = (loss_GAN_AB + loss_GAN_BA) / 2
    # Cycle loss
    recov_A = G_BA(fake_B)
    loss_cycle_A = criterion_cycle(recov_A, real_A)
    recov_B = G_AB(fake_A)
    loss_cycle_B = criterion_cycle(recov_B, real_B)
    loss_cycle = (loss_cycle_A + loss_cycle_B) / 2
    # Total loss
    loss_G = loss_GAN + lambda_cyc * loss_cycle + lambda_id *
loss_identity

    ### Test Discriminator-A
    D_A.eval()
    # Real loss
    loss_real = criterion_GAN(D_A(real_A), valid)
    # Fake loss (on batch of previously generated samples)
    fake_A_ = fake_A_buffer.push_and_pop(fake_A)
    loss_fake = criterion_GAN(D_A(fake_A_.detach()), fake)
    # Total loss
    loss_D_A = (loss_real + loss_fake) / 2

    ### Test Discriminator-B
    D_B.eval()
    # Real loss
    loss_real = criterion_GAN(D_B(real_B), valid)
    # Fake loss (on batch of previously generated samples)
    fake_B_ = fake_B_buffer.push_and_pop(fake_B)

```

```

        loss_fake = criterion_GAN(D_B(fake_B.detach()), fake)
        # Total loss
        loss_D_B = (loss_real + loss_fake) / 2
        loss_D = (loss_D_A + loss_D_B) / 2

        ### Log Progress
        loss_gen += loss_G.item(); loss_id += loss_identity.item();
        loss_gan += loss_GAN.item(); loss_cyc += loss_cycle.item()
        loss_disc += loss_D.item(); loss_disc_a += loss_D_A.item();
        loss_disc_b += loss_D_B.item()
        tqdm_bar.set_postfix(Gen_loss=loss_gen/(batch_idx+1),
                             identity=loss_id/(batch_idx+1), adv=loss_gan/(batch_idx+1),
                             cycle=loss_cyc/(batch_idx+1),
                             Disc_loss=loss_disc/(batch_idx+1),
                             disc_a=loss_disc_a/(batch_idx+1), disc_b=loss_disc_b/(batch_idx+1))

        # If at sample interval save image
        if random.uniform(0,1)<0.4:
            # Arrange images along x-axis
            real_A = make_grid(real_A, nrow=1, normalize=True)
            real_B = make_grid(real_B, nrow=1, normalize=True)
            fake_A = make_grid(fake_A, nrow=1, normalize=True)
            fake_B = make_grid(fake_B, nrow=1, normalize=True)
            # Arrange images along y-axis
            image_grid = torch.cat((real_A, fake_B, real_B, fake_A), -
1)

            save_image(image_grid, f"images/{batch_idx}.png",
normalize=False)

        test_losses_gen.append(loss_gen/len(test_dataloader))
        test_losses_disc.append(loss_disc/len(test_dataloader))

        # Save model checkpoints
        if np.argmin(test_losses_gen) == len(test_losses_gen)-1:
            # Save model checkpoints
            torch.save(G_AB.state_dict(), "saved_models/G_AB.pth")
            torch.save(G_BA.state_dict(), "saved_models/G_BA.pth")
            torch.save(D_A.state_dict(), "saved_models/D_A.pth")
            torch.save(D_B.state_dict(), "saved_models/D_B.pth")

{"model_id":"b5df0800efa9459bae7cba819a43d82d","version_major":2,"version_minor":0}

{"model_id":"aa86bc3cf2b043719f8ca907cd1b5f79","version_major":2,"version_minor":0}

{"model_id":"240f8a1e88d84adda4c9d67983b3b15f","version_major":2,"version_minor":0}

{"model_id":"99b3697eac8c4267965b6654e64e07a2","version_major":2,"version_minor":0}

```

```

{"model_id": "d260149d74674dabb0f0f2dc757bd564", "version_major": 2, "version_minor": 0}

{"model_id": "790d91feac8d4cb7991844b95f7ee341", "version_major": 2, "version_minor": 0}

{"model_id": "0aa3d19e20b54fcaa3ce19675c718bc4", "version_major": 2, "version_minor": 0}

{"model_id": "cc74927512bc47a6b99177ae04ecf08e", "version_major": 2, "version_minor": 0}

{"model_id": "1a307b1e292443539b926cf595ac4f3e", "version_major": 2, "version_minor": 0}

{"model_id": "5d3435425315422a98b830f16dff5d6b", "version_major": 2, "version_minor": 0}

{"model_id": "be8f94fade4c46c688adff056b554567", "version_major": 2, "version_minor": 0}

{"model_id": "d819e78b86064e9d83d3649babad0555", "version_major": 2, "version_minor": 0}

{"model_id": "7c74d75f12084e32a380dc16d599e22d", "version_major": 2, "version_minor": 0}

fig = go.Figure()
fig.add_trace(go.Scatter(x=train_counter, y=train_losses_gen,
mode='lines', name='Train Gen Loss (Loss_G)'))
fig.add_trace(go.Scatter(x=train_counter, y=train_losses_id,
mode='lines', name='Train Gen Identity Loss'))
fig.add_trace(go.Scatter(x=train_counter, y=train_losses_gan,
mode='lines', name='Train Gen GAN Loss'))
fig.add_trace(go.Scatter(x=train_counter, y=train_losses_cyc,
mode='lines', name='Train Gen Cyclic Loss'))
fig.add_trace(go.Scatter(x=test_counter, y=test_losses_gen,
marker_symbol='star-diamond',
marker_color='orange', marker_line_width=1,
marker_size=9, mode='markers', name='Test Gen Loss (Loss_G)'))
fig.update_layout(
width=1000,
height=500,
title="Train vs. Test Generator Loss",
xaxis_title="Number of training examples seen (A+B)",
yaxis_title="Generator Losses"),
plotly.offline.plot(fig, filename = 'plotly_gen_losses.html')
fig.show()

fig = go.Figure()
fig.add_trace(go.Scatter(x=train_counter, y=train_losses_disc,

```

```

mode='lines', name='Train Disc Loss (Loss_D)'))
fig.add_trace(go.Scatter(x=train_counter, y=train_losses_disc_a,
mode='lines', name='Train Disc-A Loss'))
fig.add_trace(go.Scatter(x=train_counter, y=train_losses_disc_b,
mode='lines', name='Train Disc-B Loss'))
fig.add_trace(go.Scatter(x=test_counter, y=test_losses_disc,
marker_symbol='star-diamond',
                        marker_color='orange', marker_line_width=1,
marker_size=9, mode='markers', name='Test Disc Loss (Loss_G)'))
fig.update_layout(
    width=1000,
    height=500,
    title="Train vs. Test Discriminator Loss",
    xaxis_title="Number of training examples seen (A+B)",
    yaxis_title="Discriminator Losses"),
plotly.offline.plot(fig, filename = 'plotly_disc_losses.html')
fig.show()

```